ADVANCED OPERATING SYSTEMS WITH UNIX

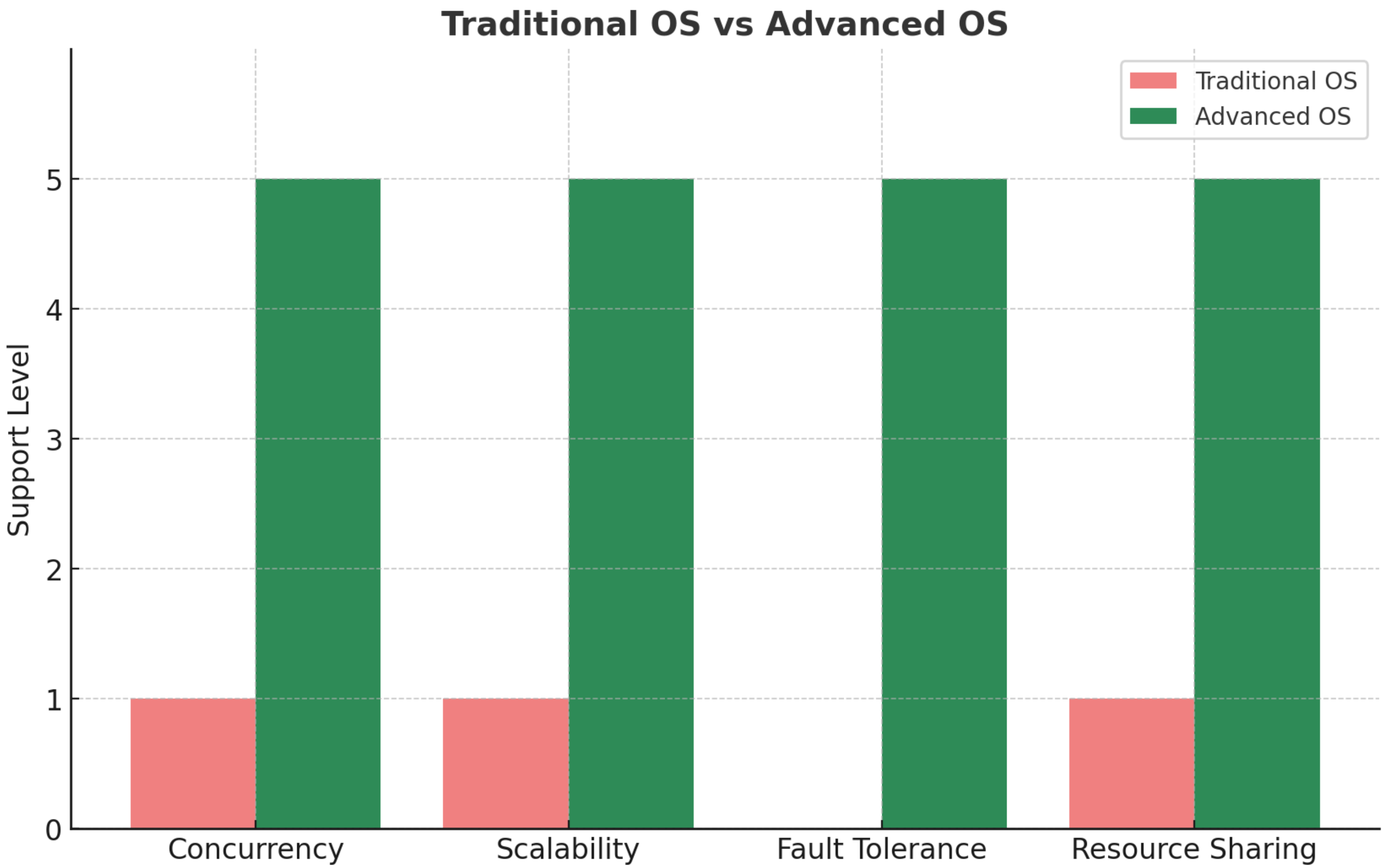
Aos- ch1-unit-1

What is an Advanced Operating System?

- Evolved form of traditional OS
- **Traditional OS**: Manages a single machine’s resources (CPU, memory, I/O).
- **Advanced OS:** Extends functionality to manage multiple processors, distributed systems, cloud resources, and parallel computations.
- **Supports complex and large-scale computing**
- Supports Complex and Large-Scale Computing Designed for supercomputers, distributed systems, clusters, and cloud computing.
- Can efficiently manage heterogeneous hardware and geographically distributed resources. Provides high performance and availability.
- *An **Advanced Operating System** (like Distributed OS, Multiprocessor OS, Real-Time OS, Cloud OS) provides **concurrency, scalability, fault tolerance, and resource sharing**, making it suitable for **modern large-scale computing**.*
- **Key Features**:
    - Concurrency
    - Scalability
    - Fault tolerance
    - Resource sharing

Key Features

1. **Concurrency**
   - Multiple processes/threads run simultaneously.
   - Supports **parallel processing, synchronization, and interprocess communication (IPC)**.
2. **Scalability**
   - Efficiently manages **growing workloads and system resources**.
   - Works well in **multiprocessor, distributed, and cloud environments**.
3. **Fault Tolerance**
   - Detects, masks, and recovers from hardware/software failures.
   - Ensures **reliability and continuous availability**.
4. **Resource Sharing**
   - Facilitates **efficient and secure sharing of CPUs, memory, storage, and networks** among multiple users or systems.
   - Often uses **virtualization and distributed resource management**.



Traditional OS vs Advanced OS
Traditional OS
Advanced OS
Support Level
0
1
2
3
4
5
Concurrency
Scalability
Fault Tolerance
Resource Sharing

Types of Advanced Operating Systems

- Distributed OS
- Multiprocessor OS
- Real-Time OS
- Cloud-Based OS

1. Distributed Operating System (DOS)

- A Distributed OS controls multiple **independent computers (nodes)** and presents them to the user as if they were a **single coherent system**.
- The main idea: **"One big virtual machine across a network"**.

How It Works

- Each node has its own local memory and CPU but runs a part of the OS.
- The system ensures **transparency**:
  - **Access Transparency**: Files/devices accessed uniformly regardless of location.
  - **Location Transparency**: User doesn't need to know where the resource is physically located.
  - **Replication Transparency**: Multiple copies of data are hidden from the user.
  - **Fault Transparency**: System hides failures and continues running.

Use Cases

- Cloud services, banking systems, stock trading systems, large-scale web apps.
- Example: **Google File System (GFS), Hadoop HDFS, Amoeba OS**.

2.Multiprocessor Operating System (MPOS)

- Runs on systems with **multiple processors tightly coupled** to a single shared memory and clock.

Two main types:

- **Asymmetric Multiprocessing (AMP)** → One master CPU controls others.
- **Symmetric Multiprocessing (SMP)** → All CPUs share equal responsibility.

How It Works

- The OS schedules tasks across multiple CPUs.
- Processes/threads can be executed **in parallel**, improving throughput.
- Provides **load balancing** so one CPU isn't overloaded.

Use Cases

- **Supercomputers, servers, parallel simulations, scientific computing**.
- Example: **Linux SMP, Windows Server, Solaris**.

3.Real-Time Operating System (RTOS)

- RTOS is used where **timing is critical**.
- The system guarantees **deterministic response times**.

Types

- **Hard RTOS**: Missing a deadline is unacceptable. (e.g., missile defense systems).
- **Soft RTOS**: Occasional missed deadlines tolerated. (e.g., video streaming).

How It Works

- Uses **priority scheduling** (Rate Monotonic Scheduling, Earliest Deadline First).
- Provides **low latency interrupt handling**.
- Has a **small, efficient kernel** (unlike general-purpose OS).

Use Cases

- Embedded systems, robotics, aerospace, automotive (engine control), medical devices.
- Example: **VxWorks, QNX, RTLinux, FreeRTOS**.

4. Cloud-Based Operating System

Concept

- Extends OS functions to **manage cloud infrastructure and services**.
- Provides an interface for **virtual machines, containers, and cloud applications**.

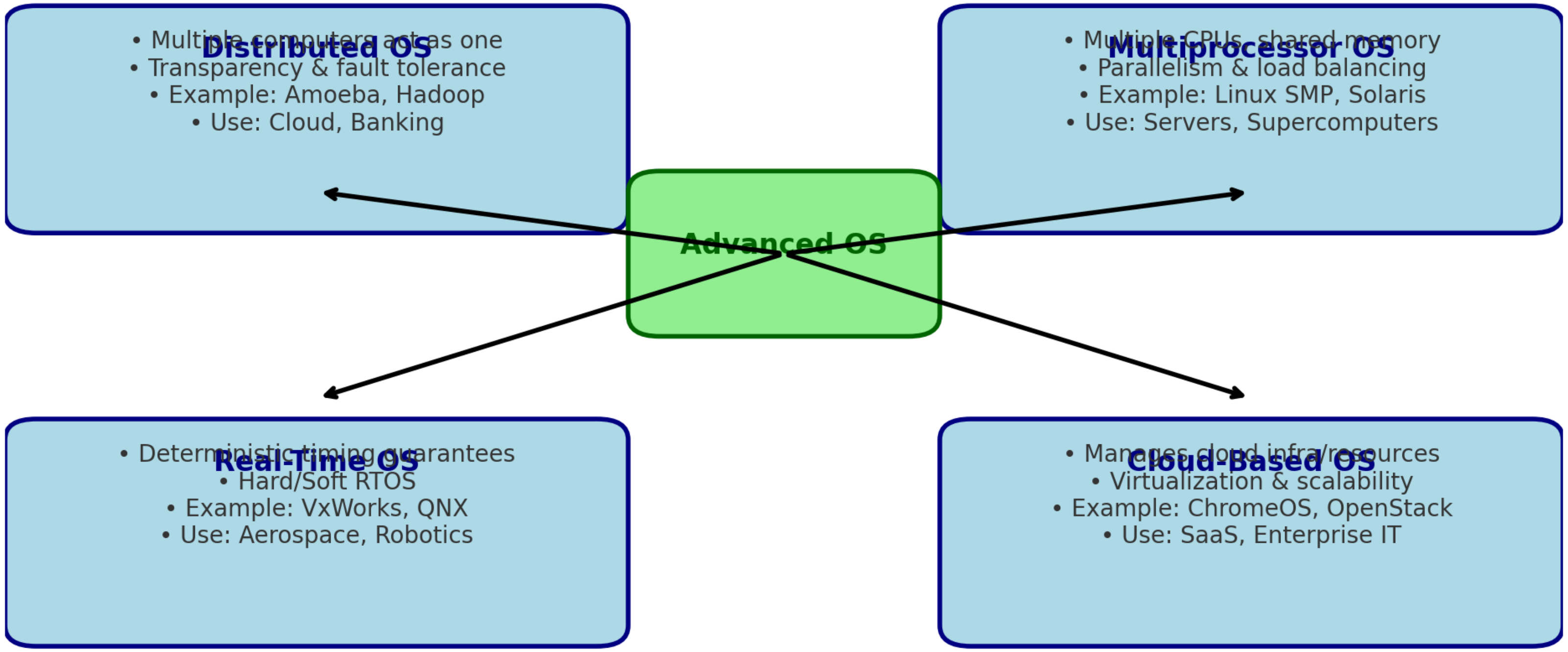
How It Works

- Uses **virtualization** (Hypervisors like VMware, KVM, Xen).
- Manages **on-demand resources** (CPU, storage, networking).
- Provides **elastic scaling** (automatically adding/removing resources).
- Ensures **multi-tenancy** (isolation between multiple users/organizations).

Use Cases

- Cloud computing, SaaS platforms, web hosting, enterprise IT.
- Example: **Google Chrome OS, OpenStack, Microsoft Azure OS, Cloud Foundry**

Types of Advanced Operating Systems



Type	Advantages	Disadvantages	Real-World Examples
Distributed OS	• High fault tolerance (system keeps running if one node fails) • Scalability (easy to add new machines) • Resource sharing across the network • Appears as a single system to users	• Complex design and management • Network dependency (failures affect performance) • Security is harder to maintain	**Amoeba OS, Hadoop HDFS, Google File System, Plan 9**
Multiprocessor OS	• High throughput via parallelism • Better CPU utilization • Load balancing across processors • Increased reliability (if one CPU fails, others work)	• Expensive hardware (multiple CPUs) • Complexity in scheduling and synchronization • Shared memory conflicts may occur	**Linux SMP, Windows Server, Solaris, IBM AIX**
Real-Time OS (RTOS)	• Predictable and deterministic execution • Low latency and fast response to events • Reliable for safety-critical tasks • Efficient resource utilization	• Limited multitasking compared to general OS • Complex to develop/debug • Requires specialized hardware/software	**VxWorks, QNX, RTLinux, FreeRTOS**
Cloud-Based OS	• Scalability & elasticity (resources scale on demand) • Cost-effective (pay-per-use model) • Resource virtualization (VMs, containers) • Multi-tenancy support	• Strong dependency on Internet • Data privacy & security concerns • Downtime affects all services	**Google ChromeOS, OpenStack, Microsoft Azure OS, Cloud Foundry**